A MINIMAL FIELD HANDBOOK

The Minimal Efficient ML Handbook

A minimal working guide to efficient ML: quantization, pruning, distillation and efficient architectures

Mosafer

V1.0 - COMPACT EDITION

NOVEMBER 2026

Contents

1 The Efficiency Landscape **2**
- 1.1 The four axes . . . 2
- 1.2 The method menu . . . 2
- 1.3 The workflow . . . 2

2 Pruning **3**
- 2.1 Unstructured: magnitude and the lottery . . . 3
- 2.2 Structured: channels and heads . . . 3
- 2.3 The theory connection . . . 3

3 Knowledge Distillation **5**
- 3.1 The classic objective . . . 5
- 3.2 The variants . . . 5
- 3.3 Distillation vs. the alternatives . . . 5

4 Efficient Architectures **7**
- 4.1 Depthwise separable convolutions . . . 7
- 4.2 Compound scaling . . . 7
- 4.3 Transformer-era efficiency . . . 7

A Cheat Sheet **9**
- A.1 The methods . . . 9
- A.2 The workflow . . . 9
- A.3 Metrics . . . 9

CHAPTER 1

The Efficiency Landscape

Efficiency is measured on four axes, and optimizing one costs the others. This chapter is the map; the Inference handbook (Ch. 4–5) covered serving, and this one covers the model itself.

1.1 The four axes

Axis	Measures
parameters	memory footprint, download size, training cost
FLOPs	compute cost per input — energy, battery
latency	wall-clock per request on the target device — the user's experience
energy	joules per inference — edge devices and climate reports

PITFALL

The axes disagree: a low-FLOPs model can be slow (unfused ops, bandwidth-bound layers — the Inference handbook's decode wall) and a low-parameter model can be energy-hungry on bad hardware. Measure on the *target device*, name the metric in every claim, and never let a paper's "efficient" mean FLOPs when your product means latency.

1.2 The method menu

Family	Chapter
quantization	compress weights/activations to 8/4 bits — the default (Inference handbook, Ch. 4; Open-Weights handbook, Ch. 4)
pruning	remove weights/structures — Ch. 2
knowledge distillation	train a student to mimic a teacher — Ch. 3
efficient architectures	design for the budget — Ch. 4
Mixture-of-Experts	capacity on demand — DL theory Ch. 6's conditional compute

1.3 The workflow

1. define the deployment target: device, latency SLO, memory budget, batch size (single-user?).
2. pick the largest model that fits, train well.
3. compress: quantize first (biggest win, least risk), then prune/distill if more is needed.
4. re-evaluate accuracy AND latency on-device at every step — the compression-evaluation loop (Statistics handbook Ch. 8's paired comparisons).

THE ONE-SENTENCE SUMMARY

Efficiency is a constrained-optimization problem (Optimization handbook, Ch. 7): maximize quality subject to the device budget — and the constraint is measured on hardware, not in a paper.

CHAPTER 2

Pruning

Pruning removes weights believed unimportant — from individual weights (unstructured) to whole channels and layers (structured). Only structured pruning speeds up real hardware; unstructured pruning compresses files.

2.1 Unstructured: magnitude and the lottery

Magnitude pruning removes the smallest-weight connections — the lottery-ticket story (DL theory, Ch. 7): train, prune the smallest weights, rewind survivors to their initialization, retrain — sparse subnetworks match the dense network.

```
import torch

def magnitude_prune(tensor, sparsity):
    k = int(tensor.numel() * sparsity)
    threshold = tensor.abs().flatten().kthvalue(k).values
    return (tensor.abs() > threshold).float()   # the binary mask
```

The catch: unstructured sparsity needs *sparse kernels* to be fast — dense hardware ignores zeros. N:M semi-structured sparsity (2:4 on NVIDIA Ampere+) is the hardware-supported compromise: every 2 of 4 weights zero, with real speedups.

2.2 Structured: channels and heads

Prune whole channels/filters/attention heads — real speedups on dense hardware, at coarser granularity:

```
import torch

scores = torch.tensor([bn.weight.abs().sum() * act_std[c]
                       for c, (bn, act_std) in enumerate(zip(bns, stds))])
drop = scores.argsort()[:n_to_remove]
# score channels by activation statistics (L1 of the BN scale,
# or Taylor expansion: |g * w| over a calibration pass)
```

Then *retrain* (fine-tune) the pruned network — pruning is a regularization surgery, not a deletion.

2.3 The theory connection

Pruning and the lottery-ticket hypothesis reframed capacity: dense networks contain sparse subnetworks that train to the same accuracy — and the Modern view: overparameterization is the *training* regime, sparsity the *deployment* regime. Prune after training (or during, gradually), not before.

> **PITFALL**
> Pruning traps: (1) pruning by magnitude across *different* layers is meaningless — prune within layers or by global-rank with per-layer scaling; (2) pruning BatchNorm-coupled channels requires updating the

running statistics — hand-rolled surgery silently breaks normalization; (3) fine-tune long enough after pruning — a pruned network without recovery training is a damaged network.

THE ONE-SENTENCE SUMMARY

Unstructured pruning compresses storage; structured pruning compresses compute — and the lottery-ticket literature explains why the pruned networks work: the dense network was one initialization away from the sparse solution anyway.

CHAPTER 3

Knowledge Distillation

Distillation trains a small student to reproduce a large teacher — not its labels, but its *function*: soft probabilities, hidden representations, reasoning traces. The teacher's dark knowledge (the relative probabilities of wrong answers) is the signal.

3.1 The classic objective

$$\mathcal{L} = (1-\lambda)\,\mathrm{CE}(y, \sigma(z_s)) + \lambda\, T^2\, \mathrm{KL}(\sigma(z_t/T) \,\|\, \sigma(z_s/T))$$

with temperature T softening both distributions (the soft targets carry inter-class structure — "a 3 looks a little like an 8"), λ balancing the hard-label loss:

```
import torch.nn.functional as F

def kd_loss(student_logits, teacher_logits, labels, T=4.0, lam=0.5):
    hard = F.cross_entropy(student_logits, labels)
    soft = F.kl_div(
        F.log_softmax(student_logits / T, dim=-1),
        F.softmax(teacher_logits / T, dim=-1),
        reduction="batchmean") * T * T
    return lam * soft + (1 - lam) * hard
```

3.2 The variants

Variant	What transfers
logit distillation	output distributions (above)
feature distillation	hidden-layer activations (attention transfer)
relational distillation	distances/relations between samples
on-policy distillation	the student's own generations scored by the teacher — RLHF-era variant (Alignment handbook, Ch. 5's DPO cousin)

```
from trl import GKDTrainer   # on-policy distillation: the student
# learns from its OWN generations, scored by the teacher (GKD)
```

3.3 Distillation vs. the alternatives

Comparison	Verdict
vs. quantization	different axes — combine both (distill, then quantize the student)
vs. training from scratch	distillation usually wins on small students — the teacher's soft targets are a curriculum
vs. fine-tuning the big model	when the budget is deployment-side, distill; when it is training-side, fine-tune

PITFALL

Distillation traps: (1) temperature is a real hyperparameter (T = 2–4 typical) — sweep it with λ jointly; (2) capacity gap — too-large teachers can teach students nothing useful (the student cannot represent the function); intermediate teachers or auxiliary losses help; (3) on-policy distillation beats offline student-data distillation for LLMs — the student learns from *its own* distribution's mistakes.

THE ONE-SENTENCE SUMMARY

Distillation transfers the teacher's *function* — softened outputs are a richer signal than labels — and the modern form distills on the student's own generations, closing the exposure-bias gap that Behavioral Cloning has (RL theory, Ch. 7).

CHAPTER 4

Efficient Architectures

When training-side compression is not enough, design for the budget. The canon: MobileNet's depthwise separable convolutions, EfficientNet's compound scaling, and the transformer-era equivalents (Grouped-Query Attention, MoE, state-space hybrids).

4.1 Depthwise separable convolutions

Standard conv: $d_k \times d_k \times C_{\text{in}} \times C_{\text{out}}$ parameters. Depthwise separable: depthwise ($d_k \times d_k \times C_{\text{in}}$) + pointwise ($1 \times 1 \times C_{\text{in}} \times C_{\text{out}}$) — roughly $1/C_{\text{out}} + 1/d_k^2$ of the FLOPs:

```
nn.Sequential(
    nn.Conv2d(C_in, C_in * 6, 3, groups=C_in),  # depthwise, expanded
    nn.BatchNorm2d(C_in * 6), nn.ReLU6(),
    nn.Conv2d(C_in * 6, C_out, 1),              # pointwise projection
)
# MobileNet block: depthwise -> pointwise -> norm/act, x expansion
```

4.2 Compound scaling

EfficientNet's insight: scale depth, width, and resolution *together* with fixed ratios — scaling one axis alone plateaus (DL theory, Ch. 5's bias–variance per axis). The compound coefficients were found once on a small baseline and transferred:

$$d = \alpha^{\phi}, \quad w = \beta^{\phi}, \quad r = \gamma^{\phi}, \qquad \alpha\beta^2\gamma^2 \approx 2$$

4.3 Transformer-era efficiency

Technique	Mechanism
GQA / MQA	fewer KV heads — inference memory (Inference handbook, Ch. 5)
sparse MoE	route tokens to k of N experts — capacity without proportional FLOPs (DL theory, Ch. 6)
state-space hybrids (Mamba-style)	linear-time sequence mixing — the attention alternative for very long contexts
sliding windows	local attention + stacked layers (Inference handbook, Ch. 5)

PITFALL

Efficient-architecture traps: (1) FLOPs are not latency — depthwise convolutions are FLOPs-cheap but memory-access-expensive (low arithmetic intensity); benchmark on-device (Ch. 1); (2) NAS-found architectures overfit their search space and hardware — transfer with care; (3) MoE training instability (router collapse, load imbalance) is real — aux load-balancing losses are standard.

THE ONE-SENTENCE SUMMARY

Efficient architectures encode the budget into the design — separable convolutions, compound scaling, GQA, MoE — while the Chapters 2–3 methods encode it into the weights. The full stack uses both, in the order that the device measurement dictates.

APPENDIX A

Cheat Sheet

A.1 The methods

Method	Cost and gain
post-training quantization	minutes of work; 4x memory; small quality risk (Inference handbook Ch. 4)
unstructured pruning	file size only; needs sparse kernels
structured (channel) pruning	real latency; needs fine-tuning
distillation	student learns the teacher's function; on-policy for LLMs
depthwise separable	$\sim 1/C_{\text{out}} + 1/d_k$ FLOPs of a standard conv
MoE	capacity at constant per-token FLOPs; routing overhead

A.2 The workflow

1. define the target device, SLO, memory, batch.
2. train the largest fitting model well.
3. quantize (PTQ) `\to` evaluate on-device `\to`
4. prune/distill if needed `\to` re-evaluate `\to`
5. repeat until quality is within tolerance at target latency.

A.3 Metrics

Axis	Measure
size	parameters, checkpoint MB
compute	FLOPs per input
speed	on-device latency at your batch size
energy	joules per inference (edge)
quality	the task metric, always paired with the axes above

> **THE ONE HABIT**
>
> Every efficiency claim carries: the device, the batch size, the metric, and the accuracy delta. "40% smaller and 2x faster with −0.3 accuracy" is an engineering result; "efficient" is an adjective.